CODESPECT

Telescope

AI ASSESSMENT REPORT

November 4th, 2026

Prepared for:

Telescope Finance

Contents

1 About CODESPECT **2**

2 Disclaimer **2**

3 Risk Classification **4**

4 Executive Summary **5**

5 Audit Summary **6**
5.1 Engagement Overview . . . 6
5.2 Scope – Audited Files . . . 6
5.3 Findings Overview . . . 7

6 Findings **8**
6.1 Medium . . . 8
6.1.1 [Medium] Immutable VPM binding with no settlement-failure handling permanently disables every sAERO exit if Aero rotates or revokes the module . . . 8
6.2 Low . . . 10
6.2.1 [Low] Dynamic-fee liquidity sample is never reconciled at launch, unpause or write-down . . . 10
6.2.2 [Low] `setVestingDuration(0)` promotes all unvested rewards without the solvency check that `pause()` applies . . . 12
6.2.3 [Low] Paused emergency cash exits are first-come at full NAV with no fee or pro-rata cap . . . 13
6.2.4 [Low] One queued request can reserve the entire master stake and delay every other holder's queued exit for a funding cycle . . . 14
6.2.5 [Low] `_mintDepositShares(...)` falls back to 1:1 pricing on zero NAV with shares outstanding, so a revival deposit is partly captured by worthless old shares . . . 15
6.2.6 [Low] A reserve-ratio increase cannot retain cash that a permissionless lock takes first; the setter does not settle outstanding excess . . . 16
6.2.7 [Low] Retrying the emergency unwind while a leaf return is in flight emits success and forfeits the caller's budget . . . 17
6.2.8 [Low] Compounders have no ERC20 rescue, rebind or partial-harvest path, so any reward token that cannot be harvested whole is stranded . . . 18
6.2.9 [Low] Compounder constructors do not cross-check the counterparties they bind immutably . . . 20
6.2.10 [Low] `disableBridgeAsset` succeeds and emits for an unconfigured key . . . 22
6.2.11 [Low] Launch safety relies on an unenforced seed and launch sequence, and the zero-supply state has two more routes than the register lists . . . 23
6.3 Informational . . . 24
6.3.1 [Info] `maxSlippageBps` does not apply to CL routes, so `tightenMaxSlippageBps(...)` cannot bound the executor on them . . . 24

7 Additional Notes **25**
7.1 Rewards recognised after a write-down also vest at the retained rate . . . 25
7.2 A shortage that appears while the vault is already paused waits for the Security Council . . . 25

1 About CODESPECT

CODESPECT is a specialised smart contract security firm dedicated to ensuring the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to building secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialise in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralised applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialised cybersecurity solutions designed to minimise risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Agentic Auditing Tool: This security review was performed by SpecSabre, an agentic auditing tool developed by CODESPECT, and not by human security auditors. Automated analysis may not identify all vulnerabilities that a manual review would identify, and the findings presented in this report may require further validation. Accordingly, this report is not a substitute for a manual security audit performed by human auditors. We recommend a manual security audit of the codebase before deployment and after any significant change to the deployed contracts.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implement-

ing a bug bounty program to incentivise external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivised.
- **Medium** - Likely only under certain conditions or moderately incentivised.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilises one other category for findings: **Informational**.

a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally, including cases where certain portions of the code deviate from established smart contract development standards or best practices.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for Telescope Finance on **Telescope**, a Liquid veAERO Vault.

The audit was performed using:

a) Automated analysis of the codebase by SpecSabre, the CODESPECT AI auditor.

CODESPECT found thirteen points of attention, one classified as `Medium`, eleven classified as `Low`, and one classified as `Informational`. All issues are summarised in the table below and detailed in Section 6.

Organisation of the document is as follows:

- **Section 3** presents the risk classification methodology and reference frameworks.
- **Section 5** summarises the audit: scope, schedule, coverage, and auditors.
- **Section 6** presents the findings.
- **Section 7** presents the additional notes raised by the auditors.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Medium	0	0	1
Low	0	0	11
Informational	0	0	1
Total	**0**	**0**	**13**

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

5.1 Engagement Overview

Audit Type	Security Review
Project Name	Telescope
Type of Project	Liquid veAERO Vault
Final Report	November 4th, 2026
Repository	Governless-Labs/telescope-public
Commit (Audit)	522843e0574253c91c193f098237c3c92a8a215f
Commit (Final)	No separate fix commit (no code changes after the audit)
Auditors	SpecSabre (CODESPECT AI auditor)

Table 3: Summary of the Audit

5.2 Scope – Audited Files

	Contract	LoC
1	LiquidityVault.sol	593
2	LeafCompounder.sol	336
3	PositionVault.sol	307
4	Router.sol	234
5	BaseCompounder.sol	217
6	libraries/MetarouterPlan.sol	172
7	interfaces/V3/IVoterV3.sol	57
8	interfaces/IMetarouter.sol	54
9	interfaces/IPositionVault.sol	43
10	interfaces/ILiquidityVault.sol	40
11	interfaces/V3/IAeroRootVoterStatus.sol	32
12	tAERO.sol	28
13	interfaces/AeroPoolInterfaces.sol	25
14	interfaces/V3/IVotingEscrowV3.sol	23
15	interfaces/V3/IVoterPaymentsModule.sol	14
16	interfaces/IRouter.sol	11
17	interfaces/ITokenRouter.sol	10
18	libraries/Errors.sol	10
19	interfaces/ItAEROToken.sol	8
20	interfaces/IWrappedNative.sol	5
	Total	**2219**

5.3 Findings Overview

	Finding	Severity	Update
1	[Medium] Immutable VPM binding with no settlement-failure handling permanently disables every sAERO exit if Aero rotates or revokes the module	Medium	Acknowledged
2	[Low] Dynamic-fee liquidity sample is never reconciled at launch, unpause or write-down	Low	Acknowledged
3	[Low] `setVestingDuration(0)` promotes all unvested rewards without the solvency check that `pause()` applies	Low	Acknowledged
4	[Low] Paused emergency cash exits are first-come at full NAV with no fee or pro-rata cap	Low	Acknowledged
5	[Low] One queued request can reserve the entire master stake and delay every other holder's queued exit for a funding cycle	Low	Acknowledged
6	[Low] `_mintDepositShares(...)` falls back to 1:1 pricing on zero NAV with shares outstanding, so a revival deposit is partly captured by worthless old shares	Low	Acknowledged
7	[Low] A reserve-ratio increase cannot retain cash that a permissionless lock takes first; the setter does not settle outstanding excess	Low	Acknowledged
8	[Low] Retrying the emergency unwind while a leaf return is in flight emits success and forfeits the caller's budget	Low	Acknowledged
9	[Low] Compounders have no ERC20 rescue, rebind or partial-harvest path, so any reward token that cannot be harvested whole is stranded	Low	Acknowledged
10	[Low] Compounder constructors do not cross-check the counterparties they bind immutably	Low	Acknowledged
11	[Low] `disableBridgeAsset` succeeds and emits for an unconfigured key	Low	Acknowledged
12	[Low] Launch safety relies on an unenforced seed and launch sequence, and the zero-supply state has two more routes than the register lists	Low	Acknowledged
13	[Info] `maxSlippageBps` does not apply to CL routes, so `tightenMaxSlippageBps(...)` cannot bound the executor on them	Info	Acknowledged

6 Findings

6.1 Medium

6.1.1 [Medium] Immutable VPM binding with no settlement-failure handling permanently disables every sAERO exit if Aero rotates or revokes the module

File(s): PositionVault.sol, PositionVault.sol, Router.sol, LiquidityVault.sol

Description: The PositionVault contract holds the master veNFT and is the only contract that can cut a fresh sAERO out of it. Both the queued claim path (withdrawFreshVeNFT(...), called by LiquidityVault.finalizeVeNFTWithdrawal(...)) and the paused exit path (emergencyWithdrawVeNFT(...), called by LiquidityVault.emergencyExit(...)) funnel into _withdrawFreshVeNFT(...), whose only stake-reducing action is a call into Aero's VoterPaymentsModule. The module address is an immutable set in the constructor, and the operator approval it needs on the VotingEscrow is granted once there as well:

```
IVoterPaymentsModule public immutable voterPaymentsModule; // @audit no setter, no re-approval path
// ...
constructor(...) {
    // ...
    voterPaymentsModule = IVoterPaymentsModule(_voterPaymentsModule);

    votingEscrow.setApprovalForAll(_voterPaymentsModule, true); // @audit granted once, only here
    aeroToken.forceApprove(_votingEscrow, type(uint256).max);
}
// ...
function _withdrawFreshVeNFT(address _recipient, uint256 _grossAmount)
    internal returns (uint256 tokenId, uint256 actualAmount)
{
    require(_grossAmount <= lockedAmount(), "Exceeds master stake");
    // ...
    // @audit the single call every sAERO exit depends on; a revert here has no handling anywhere
    uint256[] memory mintedIds = voterPaymentsModule.withdrawToNFT(masterTokenId, destinations);
    require(mintedIds.length == 1, "Unexpected VPM result");
    // ...
}
```

However, the module's standing is controlled by Aero governance, not by Telescope. If Aero revokes the module's VPM_ROLE on the VotingEscrow, or restricts the withdrawToNFT selector for an unregistered caller, withdrawToNFT(...) reverts and no Telescope function can rebind a successor module or re-approve it. Because the master NFT is owned by the immutable PositionVault and no function transfers it out, a Telescope redeployment does not recover the stake either. The deployment runbook (docs/DEPLOYMENT.md) already instructs the operator to recheck VPM authorisation after any Aero governance change, which shows the dependency is anticipated but not handled in code. The failure is also not treated as a breaker. Router.finalizeVeNFTWithdrawal(...) bubbles every revert except CircuitBreakerInvariant, so the vault stays unpaused:

```
function finalizeVeNFTWithdrawal(uint256 _requestId) external nonReentrant returns (uint256 tokenId) {
    try liquidityVault.finalizeVeNFTWithdrawal(_requestId) returns (uint256 _tokenId) {
        tokenId = _tokenId;
    } catch (bytes memory reason) {
       if (!_isCircuitBreaker(reason)) _bubbleRevert(reason); // @audit a VPM revert is bubbled, vault stays unpaused
        if (!liquidityVault.paused()) liquidityVault.pause();
        emit CircuitBreakerTriggered(msg.sender);
    }
}
```

In addition, the recipient of a funded, solvent, non-stale request cannot release it while the vault is unpaused:

```
function cancelVeNFTWithdrawal(uint256 _requestId) external nonReentrant {
    require(isVeNFTWithdrawalReady(_requestId), "Request not ready");
    VeNFTWithdrawalRequest memory request = withdrawalRequest[_requestId];
    require(
        (paused() && msg.sender == request.recipient) // @audit needs a pause or staleness; a dead VPM gives neither
            || request.grossAmount > convertToAssets(request.tAEROToBurn),
        Unauthorized()
    );
    // ...
}
```

The open reservation keeps counting in `totalPendingWithdrawals()`, which blocks new requests (`"Exceeds master stake"` in `requestVeNFTWithdrawal(...)`) and paused sAERO exits (`"CHAIN0 below queued claims"` in `emergencyWithdrawVeNFT(...)`). The repository's own `WithdrawalLiveness.integration.test.ts` drives the role revocation case and shows finalisation reverting with `NotVoterPaymentsModule`.

Impact: After an Aero governance action on the VoterPaymentsModule, the master stake (kept at about 95 percent of NAV by the excess-lock policy) becomes permanently uncuttable in sAERO form. Queued requests stay open, their escrowed tAERO is released only after the Security Council pauses, and every other holder's queued or paused sAERO exit is blocked by the dead reservation. Cash exits keep working up to the liquid reserve and no principal is destroyed, but the only way to realise the locked majority of backing is lost for good. The trigger is an external Aero governance change, not a Telescope role acting wrongly.

Recommendation(s): Treat a VPM settlement revert as a breaker: either pause the vault from the Router's catch branch, or flag the request so that its recipient may cancel it without a pause. Give the governance Timelock a path to rebind the VoterPaymentsModule and re-grant `setApprovalForAll(...)` to a successor module, applying the same reciprocal checks the deployment runbook prescribes. If the immutable binding is kept by design, document explicitly that an Aero VPM change makes the master stake permanently unrecoverable in sAERO form and that the operational response is pause, cancel and cash exits.

Status: Acknowledged

Update from the client: We acknowledge that sAERO settlement depends on continued compatible Aero VPM support and will make that dependency and the pause/cancel response explicit. We do not propose treating arbitrary settlement reverts as a circuit breaker because public invalid requests could trigger a pause. We intend to retain the immutable VPM because a migration mechanism would introduce additional operator and custody authority. The owner reports no indication of a planned Aero migration, but this is not a guarantee of continued compatibility.

6.2 Low

6.2.1 [Low] Dynamic-fee liquidity sample is never reconciled at launch, unpause or write-down

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`, `LiquidityVault.sol`, `LiquidityVault.sol`, `Router.sol`

Description: Cash redemptions through `Router.redeemAERO(...)` pay a dynamic fee that `quoteRedeemAERO(...)` computes from the live available liquidity `L` and a delayed liquidity sample `S` (`epochLiquiditySnapshot()`). The sample is a two-generation smoothing window: `pokeSnapshot()` promotes the previously stored sample and records the current liquidity as the next one, once per `snapshotInterval`. The three sample slots are written only by the constructor and by `pokeSnapshot()`:

```
constructor(...) {
    // ...
    _epochLiquiditySnapshot = MIN_LIQUIDITY_FLOOR; // @audit both generations seeded with the 1,000 AERO constant
    nextEpochLiquiditySnapshot = MIN_LIQUIDITY_FLOOR;
    lastSnapshotTime = block.timestamp;
}
// ...
function pokeSnapshot() public {
    if (block.timestamp < lastSnapshotTime + snapshotInterval) return;

    // Release before sampling so the next delayed sample reflects real cash.
    _releaseFeeBacking();
    _epochLiquiditySnapshot = nextEpochLiquiditySnapshot; // @audit the promoted value is always one generation old
    uint256 currentLiquidity = getAvailableLiquidity();
    nextEpochLiquiditySnapshot = currentLiquidity > MIN_LIQUIDITY_FLOOR
        ? currentLiquidity : MIN_LIQUIDITY_FLOOR;
    lastSnapshotTime = block.timestamp;
}
```

However, none of the lifecycle transitions that move liquidity in a step reconcile the sample, so the first exits after each transition are priced against a reference that does not describe the vault. At launch, `bootstrapMasterVeNFT(...)` pokes while liquidity is zero, and `depositAERO(...)` pokes before crediting the deposit, so the first due promotion stores the pre-deposit floor again. For one to two intervals after public cash arrives, `quoteRedeemAERO(...)` prices exits with `S = 1,000 AERO` against a live `L` that may be tens of thousands of AERO:

```
function quoteRedeemAERO(uint256 _tAEROAmount) public view
    returns (uint256 baseFeeAmount, uint256 dynamicFeeAmount, uint256 payoutAmount)
{
    // ...
    uint256 available = liquidityVault.getAvailableLiquidity();
    uint256 snapshot = liquidityVault.epochLiquiditySnapshot(); // @audit stale or seeded reference after a transition
    // ...
    dynamicFeeAmount = liquidityVault.convertToSharesRoundUp(
        _calculateDynamicFee(available, assets, snapshot)
    );
    // ...
}
```

With the launch parameters of the fee curve, a near-total cash exit in that window pays close to zero dynamic fee (about 2.7 AERO on a 50,000 AERO exit, versus about 5,060 AERO against a measured reference). The mirror case occurs after an incident. `pause()` and `unpause()` touch vesting and held fee backing but never the sample, `emergencyExit(...)` pays out liquidity without poking, and `_writeDownShortage(...)` reduces `totalUserDeposits` and the reward pools without poking:

```
function pause() external {
    // ...
    _updateVesting();
    _releaseFeeBacking(); // @audit sample slots untouched
    // ...
    _pause();
}

function unpause() external onlyGovernance {
    _unpause(); // @audit no resample at the natural reconciliation point
}
// ...
function _writeDownShortage(uint256 _accounted, uint256 _actual) internal {
    // ...
    totalProtocolRewards -= fromRewards;
    // ...
    totalUserDeposits = shortage - fromRewards >= totalUserDeposits
        ? 0
        : totalUserDeposits - (shortage - fromRewards); // @audit liquidity drops, sample keeps the pre-loss value
    emit ShortageWrittenDown(shortage);
}
```

The first `redeemAERO(...)` after `unpause()` promotes the pre-event sample, so a residual holder is charged as if liquidity had just collapsed from `S` to `L`. Our reproduction of the paused-drain case measured about 7,071 AERO of fee versus about 1,179 AERO against a reconciled sample at `L / S = 0.1`, and the write-down case measured about 525 AERO versus about 3.4 AERO. The write-down leg is acknowledged in the TEL-10 remediation record as a "subsequent dynamic-fee reference effect"; the launch leg and the paused-drain leg are not documented, and TEL-4 documents the constant seed only in the expensive direction. The fee is shown by `quoteRedeemAERO(...)` and bounded by `_minAEROToReceive`, so exposure is voluntary, and the fee is retained by remaining holders rather than lost.

Impact: For one to two snapshot intervals after launch, after an unpause or after a write-down, cash exits are mispriced in either direction: launch exits are almost fee-free, and post-incident exits are charged a fee that reflects a liquidity drop the vault already absorbed. The mispricing is socialised to remaining holders and bounded by one window; no principal is lost.

Recommendation(s): Resample at every lifecycle transition: set both `_epochLiquiditySnapshot` and `nextEpochLiquiditySnapshot` to `getAvailableLiquidity()` and `lastSnapshotTime` to `block.timestamp` in `unpause()` and at the end of `_writeDownShortage(...)`. At launch, either apply the same reset on the first public deposit after bootstrap or seed the sample from the post-bootstrap cash rather than the constant.

Status: Acknowledged

Update from the client: We agree that the delayed reference can differ substantially from current liquidity, including after startup and incidents, and will clarify this behavior and its call-dependent duration. The lag is part of the smoothing design rather than inherently an incorrect price. Please provide the exact inputs and reproduction for the numerical examples so we can compare the curves under the same conditions.

6.2.2 [Low] `setVestingDuration(0)` promotes all unvested rewards without the solvency check that `pause()` applies

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`

Description: Protocol rewards forwarded to the `LiquidityVault` vest linearly over `vestingDuration` before they count as liquid in `getAvailableLiquidity()`. Two paths release the whole unvested pool at once. `pause()`, callable by the Security Council, the Timelock or the Router, promotes it only when the vault's AERO balance covers everything it has booked, and `checkSolvency()` documents why: releasing unbacked rewards makes an accounting deficit worse.

```
function pause() external {
    // ...
    _updateVesting();
    _releaseFeeBacking();
    if (aeroToken.balanceOf(address(this)) >= _accountedAERO()) { // @audit promotion gated on solvency
        vestedRewards = totalProtocolRewards;
        rewardRate = 0;
        periodFinish = block.timestamp;
    }

    _pause();
}
```

However, the zero branch of `setVestingDuration(...)`, callable by the governance Timelock, performs the identical promotion unconditionally. It has no `whenNotPaused` modifier and the promotion is one-way:

```
function setVestingDuration(uint256 _newVestingDuration) external onlyGovernance {
    require(_newVestingDuration <= 30 days, InvalidParam());
    _updateVesting();
    if (_newVestingDuration == 0) {
        vestedRewards = totalProtocolRewards; // @audit same promotion, no balanceOf >= _accountedAERO() check
        rewardRate = 0;
        periodFinish = block.timestamp;
    }
    vestingDuration = _newVestingDuration;
    emit VestingDurationUpdated(_newVestingDuration);
}
```

If the write lands while the vault carries a backing shortfall that has not yet been crystallised by `checkSolvency()`, `getAvailableLiquidity()` rises by the whole unvested pool. Cash exits (`withdrawAERO(...)`, or `emergencyExit(...)` while paused) then pay recorded-NAV claims out of the surviving AERO, and the holders who exit after the write-down recover less. The promoted figure also feeds the dynamic-fee inputs. No production path creates such a shortfall; our reproduction injects the loss with the test-only `forceTransferAero` hook and shows a slow holder recovering about 1,000 AERO instead of about 14,500 AERO.

Impact: A governance write to `setVestingDuration(0)` during an uncrystallised backing loss redistributes the shortage from early exiters to late exiters, bounded by the size of the shortage. The precondition requires an exogenous loss and a trusted governance action, so the issue is a code-level inconsistency between two release paths rather than a parameter choice.

Recommendation(s): Guard the zero-duration promotion with the same `aeroToken.balanceOf(address(this)) >= _accountedAERO()` test that `pause()` applies, or revert when a material shortage is detectable, and consider adding `whenNotPaused` to the setter.

Status: Acknowledged

Update from the client: We agree that the zero-duration path can promote unbacked rewards during an existing shortage. Our operational policy is to set duration to zero only after verifying restoration or reconciliation of backing and not to unpause while a shortage remains unresolved. This policy is not enforced by an on-chain guard, and paused emergency exits can still be priced using stale NAV.

6.2.3 [Low] Paused emergency cash exits are first-come at full NAV with no fee or pro-rata cap

File(s): `LiquidityVault.sol`

Description: While the vault is paused, any tAERO holder can call `emergencyExit(...)` directly on the `LiquidityVault` and choose between a cash payout (`WithdrawalChoice.AERO`) and a fresh sAERO cut from the master stake (`WithdrawalChoice.VENFT`). The AERO branch prices the claim at `convertToAssets(...)`, caps it only at the total available liquidity and pays through `_transferLiquidAERO(...)` with no base fee, no dynamic fee and no VPM fee:

```
function emergencyExit(uint256 _tAEROAmount, WithdrawalChoice _choice, uint256 _minAEROOut)
    external whenPaused nonReentrant
{
    // ...
    if (_choice == WithdrawalChoice.AERO) {
        _updateVesting();
        uint256 claim = convertToAssets(_tAEROAmount); // @audit full NAV, no fee
        uint256 payout = claim;
        if (payout > getAvailableLiquidity()) payout = getAvailableLiquidity(); // @audit cap is the whole pool
        require(payout > 0, "No liquidity");
        require(payout >= _minAEROOut, SlippageExceeded());
        // ...
        _transferLiquidAERO(msg.sender, payout);
        emit EmergencyExitExecuted(msg.sender, _choice, 0, tAEROBurn, payout);
        return;
    }

    uint256 grossAmount = convertToAssets(_tAEROAmount);
    // ...
    // @audit later exiters land here and receive net of Aero's live VPM fee
    (uint256 tokenId, uint256 actualAmount) = positionVault.emergencyWithdrawVeNFT(msg.sender, grossAmount);
    require(actualAmount >= _minAEROOut, SlippageExceeded());
    // ...
}
```

However, the liquid pool is finite and smaller than the aggregate claims whenever the master stake holds most of the backing, so it is allocated by transaction order. Whoever calls first takes the full cash claim; later exiters either revert with `"No liquidity"` or must take the VENFT branch, which delivers an sAERO net of Aero's live VPM withdrawal fee (`PositionVault._quoteVPM(...)` and `_withdrawFreshVeNFT(...)`). A front-runner can also push a specific victim's payout below its `_minAEROOut`. The first exiter additionally escapes the dynamic fee that an unpaused exit of the same size would have paid, which is the loss the pause itself creates (see the issue on the unreconciled liquidity sample). The window opens only through an authorised pause or a real shortage, and both exiters still receive full value in one form or the other. The production VPM rate is not fixed in the repository; our reproduction pins a 25 percent rate in the mock and shows a 400,000 versus 375,000 incidence between two equal holders.

Impact: During a pause, the realised value of equal claims depends on transaction ordering: the ordering shifts up to `vpmFee x availableLiquidity` between exiters, and a holder can be forced onto the VPM-charged sAERO branch or below its slippage bound. No principal is lost.

Recommendation(s): Cap each emergency cash payout at the caller's pro-rata share of the available liquidity, for example `claim * getAvailableLiquidity() / getTotalVaultValue()`, with the remainder served through the sAERO branch. Alternatively, document explicitly that paused cash exits are first-come and bounded by the liquid reserve.

Status: Acknowledged

Update from the client: We acknowledge the ordering and potential difference in external VPM fee incidence between emergency exit choices. First-come cash access is the intended emergency policy, and we will document its limits and the possibility that later users must use the locked-asset path. A pro-rata mechanism would require a broader design for repeated calls and unavailable locked settlement.

6.2.4 [Low] One queued request can reserve the entire master stake and delay every other holder's queued exit for a funding cycle

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`, `PositionVault.sol`, `PositionVault.sol`, `PositionVault.sol`

Description: A tAERO holder who wants to exit into a fresh sAERO calls `Router.requestVeNFTWithdrawal(...)`, which escrows the tAERO in the vault and books a gross reservation against the master stake through `LiquidityVault.requestVeNFTWithdrawal(...)`. The reservation is counted in `totalPendingWithdrawals()` until the request is closed by `finalizeVeNFTWithdrawal(...)` or `cancelVeNFTWithdrawal(...)`.

```
function requestVeNFTWithdrawal(
    address _recipient, uint256 _netAmount, uint256 _grossAmount, uint256 _tAEROToBurn
) external whenNotPaused onlyRouter returns (uint256 requestId) {
    require(_grossAmount >= MIN_WITHDRAWAL_REQUEST_AMOUNT, BelowMinWithdrawalRequest());
    // @audit the only admission test; one request may reserve the whole master stake
    require(totalPendingWithdrawals() + _grossAmount <= positionVault.lockedAmount(), "Exceeds master stake");
    // ...
}
```

The admission test above is the only bound on a request: there is no per-recipient limit and no cap below `positionVault.lockedAmount()`. A single holder with tAERO worth close to the vault NAV can therefore reserve the entire master stake. While that reservation is open, every other holder's `requestVeNFTWithdrawal(...)` reverts with `Exceeds master stake`, paused sAERO exits revert in `PositionVault.emergencyWithdrawVeNFT(...)`, and the allocation controller cannot deploy new CHAIN0 power because `_checkBatchAllocation(...)` subtracts the same pending figure from the free CHAIN0 balance.

```
function emergencyWithdrawVeNFT(address _recipient, uint256 _grossAmount)
    external onlyLiquidityVault returns (uint256 tokenId, uint256 actualAmount)
{
    // @audit the open reservation counts against every paused sAERO exit
    require(
        chain0Balance() >= liquidityVault.totalPendingWithdrawals() + _grossAmount, "CHAIN0 below queued claims"
    );
    // ...
}
```

The block is bounded by the funding cycle rather than open-ended. Finalisation is permissionless through `Router.finalizeVeNFTWithdrawal(...)`, and `LiquidityVault.finalizeVeNFTWithdrawal(...)` carries no `whenNotPaused`, so any affected holder can close a funded request at the reserver's expense (the reserver pays Aero's VPM fee on the whole reserved gross). For an unfunded request the controller must deallocate the shortfall on its next payload, and once the request is 14 days old anyone can force the unwind through `PositionVault.emergencyUnwindMasterAllocations(...)`.

Impact: One holder can delay every other holder's queued sAERO exit and idle the vault's CHAIN0 power for the duration of a funding cycle. Affected holders keep their tAERO and the AERO cash exit, and each cycle costs the reserver the VPM fee on the reserved gross, so no claim is lost.

Recommendation(s): Document that any funded request may be finalised by third parties to release its reservation, and consider bounding a single request, for example to the free CHAIN0 balance or to a fraction of `positionVault.lockedAmount()`.

Status: Acknowledged

Update from the client: We agree that a sufficiently large, fully escrowed request can reserve available master capacity and delay subsequent requests. The requester must commit a correspondingly large economic claim, and the reservation supports asynchronous settlement. We will clarify the priority and keeper response rather than introduce an address-based cap.

6.2.5 [Low] `_mintDepositShares(...)` falls back to 1:1 pricing on zero NAV with shares outstanding, so a revival deposit is partly captured by worthless old shares

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`

Description: Every tAERO mint is priced by `_mintDepositShares(...)`: `depositAERO(...)` and `depositVeNFTs(...)` through the Router, and `bootstrapMasterVeNFT(...)` through governance. The function receives the vault value and the share supply recorded before the deposit and converts the deposited amount into shares.

```
function _mintDepositShares(
    uint256 _amount,
    uint256 _valueBefore,
    uint256 _supplyBefore,
    address _recipient
) internal returns (uint256 shares) {
    require(_recipient != address(0), ZeroAddress());

    // @audit 1:1 branch is also taken when shares are outstanding but the NAV is zero
    shares = _supplyBefore == 0 || _valueBefore == 0
        ? _amount
        : (_amount * _supplyBefore) / _valueBefore;
    // ...
    tAEROToken.mint(_recipient, shares);
}
```

The 1:1 branch is keyed on `_supplyBefore == 0 || _valueBefore == 0`. The second condition can hold while shares are still outstanding: `getTotalVaultValue()` is a sum of non-negative terms, so a full write-down of the liquid AERO with `positionVault.lockedAmount() == 0` leaves `S > 0` shares against a NAV of zero. If governance then unpauses and a depositor brings `A` AERO, that depositor receives `A` shares, but those shares are worth only `A * A / (S + A)` because the wiped shares are revalued by the deposit. The Router's `_minShares` guard cannot protect the depositor, since `convertToSharesRoundDown(...)` and `convertToSharesRoundUp(...)` repeat the same branch and quote `A` shares.

```
function convertToSharesRoundDown(uint256 _assets) public view returns (uint256) {
    uint256 supply = tAEROToken.totalSupply();
    uint256 value = getTotalVaultValue();
    // @audit same fallback; a pre-deposit quote returns the full amount
    if (supply == 0 || value == 0) return _assets;
    return (_assets * supply) / value;
}
```

The same branch prices a post-loss bootstrap and a post-loss sAERO deposit. For comparison, OpenZeppelin's ERC4626 (v5.4) has no such fallback: it adds virtual shares and a virtual asset to both sides of the conversion, so a deposit into a vault with outstanding shares and zero assets is priced against one virtual asset, and the depositor keeps almost all of the deposited value. The precondition is a total loss of backing, which no production path creates, followed by a governance unpause and a fresh deposit.

Impact: After a total loss, the first revival deposit is split with shares that no longer have any backing, and the depositor cannot defend against the split with the Router's slippage parameter.

Recommendation(s): Key the 1:1 branch on `_supplyBefore == 0` only and revert when `_valueBefore == 0` while shares are outstanding. Mirror the same rule in `convertToSharesRoundDown(...)` and `convertToSharesRoundUp(...)`.

Status: Acknowledged

Update from the client: We agree that zero NAV with outstanding shares needs explicit treatment, including when the retained seed survives a total loss. We will not reopen a completely wiped vault under its existing shares; recovery or replacement requires separate review, and reconciliation to zero NAV alone does not make it restartable. We will not unpause with an unresolved shortage. These are operational policies, not on-chain guards. The OpenZeppelin v5.4 comparison should account for its virtual assets and shares.

6.2.6 [Low] A reserve-ratio increase cannot retain cash that a permissionless lock takes first; the setter does not settle outstanding excess

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`, `LiquidityVault.sol`

Description: `minLiquidBps` defines the share of vault value that stays liquid for AERO cash exits. Any account may call `lockExcessAERO(...)`, which moves the liquid AERO above that reserve into the master NFT; the resulting stake is permanent because `PositionVault.increaseMasterStake(...)` adds it to a permanent lock and nothing cuts the master except the sAERO exit paths.

```
function lockExcessAERO() external nonReentrant whenNotPaused withCircuitBreaker returns (uint256 locked)
{
    // ...
    // @audit the excess is priced with the ratio stored at call time
    locked = getExcessLiquidAERO();
    if (locked == 0) return 0;

    _consumeLiquidBacking(locked);
    aeroToken.safeTransfer(address(positionVault), locked);
    positionVault.increaseMasterStake(locked);
    lastExcessLockAt = block.timestamp;
}
```

Governance changes the ratio with `setMinLiquidBps(...)`, which is a bare store.

```
function setMinLiquidBps(uint256 _newMinLiquidBps) external onlyGovernance {
    require(_newMinLiquidBps <= 10_000, InvalidParam());
    // @audit no effective time and no settlement of excess already above the old target
    minLiquidBps = _newMinLiquidBps;
    emit MinLiquidBpsUpdated(_newMinLiquidBps);
}
```

Because the setter neither schedules the new ratio nor settles the cash that the old ratio already classes as excess, any account can execute `lockExcessAERO(...)` in the block before a scheduled increase lands and lock the cash the increase was meant to keep liquid. Once locked, that cash is master stake; cash exits above the old reserve revert until liquidity rebuilds and the documented dynamic-fee regime applies meanwhile. NAV is unchanged. At the zero-second launch Timelock delay the window is a mempool race, and governance can already close it by batching `pause()`, `setMinLiquidBps(...)` and `unpause()` atomically.

Impact: Cash that a reserve increase intended to keep available for AERO exits can be turned into permanent master stake by a front-running lock, leaving cash exits above the old reserve unavailable until liquidity rebuilds. No value is lost.

Recommendation(s): Make `setMinLiquidBps(...)` pause-aware or batch ratio increases with `pause()` and `unpause()`. Alternatively, store a pending ratio so that `getExcessLiquidAERO()` prices the reserve against `max(old, pending)` until the new ratio takes effect, and document that a ratio increase is forward-looking only.

Status: Acknowledged

Update from the client: We acknowledge that a lock can be ordered before a reserve increase and will use the then-current ratio. The increase applies prospectively and does not unwind an earlier lock. We will document this operational limitation; batching the update cannot prevent a preceding transaction.

6.2.7 [Low] Retrying the emergency unwind while a leaf return is in flight emits success and forfeits the caller's budget

File(s): `PositionVault.sol`

Description: The `emergencyUnwindMasterAllocations(...)` function is the permissionless recovery path that pulls the master NFT's stake back from selected leaf chains, either during a pause or for an overdue unfunded withdrawal request. For every chain the caller names, it reads the amount Aero's root `Voter` currently books for that chain and ships one `DEALLOC_GAUGE` entry of exactly that size, paid for with the caller's native transport budget:

```
function emergencyUnwindMasterAllocations(
    uint256 _requestId, GaugeAllocationDispatch[] calldata _chainParams
) external payable nonReentrant {
    // ...
    for (uint256 i; i < _chainParams.length; ++i) {
        // ...
        // @audit the root booking is read; it is not reduced until the leaf's return lands
        uint128 currentAmount = rootVoter.allocationChainAmounts(tokenId, params.chainId);
        require(currentAmount > 0, "Nothing allocated");

        GaugeAllocation[] memory gauges = new GaugeAllocation[](1);
        // @audit the full root booking is dispatched even if a prior deallocation is still in flight
        gauges[0] = GaugeAllocation({gauge: DEALLOC_GAUGE, allocated: currentAmount, data: bytes("")});
        // ...
    }

    require(msg.value == expectedValue, InvalidValue());

    rootVoter.allocate{value: msg.value}(
        tokenId,
        new ChainAllocationDispatch[](0),
        dispatches,
        msg.sender
    );
}
```

Root and leaf accounting are asynchronous: the root booking is only lowered when the leaf's `Deallocate` return is delivered, while the leaf reduces its own budget as soon as the first deallocation arrives. The root-side validation compares the dispatched amount with the same root booking the function read, so a second call made while the first return is still in flight is accepted at root, emits `GaugesAllocated`, and charges the caller's `deallocationReturnCost` and transport fee. The leaf then compares the dispatched amount against its already-reduced budget and rejects the message with `ChainAllocationMismatch`. The repository's own `WithdrawalLiveness.integration.test.ts` pins this mechanism. A keeper that retries on a stale view therefore pays for a dispatch that cannot succeed and sees a success event for it; the recovery itself still completes through the in-flight return.

Impact: The retrying caller loses its deallocation return cost and transport fee and receives a misleading `GaugesAllocated` event. No holder funds are at risk, since the first dispatch still returns the stake.

Recommendation(s): Record a per-chain in-flight marker (chain id plus the root booking at dispatch) and refuse a repeat dispatch for that chain until the root booking changes. Alternatively, document for keepers how to detect an in-flight return from the root `Voter` events before calling `emergencyUnwindMasterAllocations(...)` again.

Status: Acknowledged

Update from the client: We agree that a retry can be submitted before root state reflects an in-flight return and that a dispatch event does not establish delivery. We will update keeper procedures to track messages and reconcile delivery before retrying. Costs depend on the actual dispatch and refund behavior, and a permanent in-flight lock could itself impair recovery.

6.2.8 [Low] Compounders have no ERC20 rescue, rebind or partial-harvest path, so any reward token that cannot be harvested whole is stranded

File(s): `BaseCompounder.sol`, `BaseCompounder.sol`, `LeafCompounder.sol`, `LeafCompounder.sol`, `LeafCompounder.sol`

Description: `BaseCompounder` and `LeafCompounder` receive the master NFT's reward tokens and convert them into AERO through Aero's MetaRouter. The `ROUTE_EXECUTOR_ROLE` calls `swapToAERO(...)` on Base and `harvestToBase(...)` or `harvestAndRedeem(...)` on a leaf; each call sells the complete held balance of `tokenIn`. The MetaRouter is the only outbound path for a non-AERO token: neither contract has a rescue, sweep or rebind function, and the `metarouter` and `factoryRegistry` references are immutable. Three independent conditions make a token impossible to harvest whole, and because there is no other exit, each of them strands that token's inventory permanently. First, every dispatch requires the bound MetaRouter to still be approved by Aero's factory registry:

```
function _execute(bytes memory commands, bytes[] memory inputs, uint256 deadline, uint256 value) internal {
    // @audit an Aero delisting of the immutable router freezes every non-AERO balance held at that time
    require(factoryRegistry.isMetaRouterApproved(address(metarouter)), MetarouterNotApproved());
    // ...
}
```

If Aero governance delists or replaces the MetaRouter, the compounders can no longer move any reward token and must be redeployed, which does not recover what they already hold. Second, every sale is sized on the full reported `balanceOf` and the V2 price floor is derived from that nominal amount. `BaseCompounder.swapToAERO(...)` and `LeafCompounder._baseBatch(...)` share the pattern:

```
function swapToAERO(address tokenIn, MetarouterPlan.Route calldata route, uint256 deadline)
    // ...
{
    require(tokenIn != address(aeroToken), InvalidRoute());
    // @audit the whole reported balance is sold; no caller-chosen amount is possible
    uint256 amountIn = IERC20(tokenIn).balanceOf(address(this));
    require(amountIn > 0, ZeroAmount());
    _validateSwap(tokenIn, address(aeroToken), route);

    // @audit the floor is quoted on the nominal amount, so a short delivery cannot clear it
    uint256 minAmountOut = _minimumOut(tokenIn, address(aeroToken), route, amountIn);
    IERC20(tokenIn).forceApprove(address(metarouter), amountIn);
    // ...
}



function _baseBatch(address tokenIn, address tokenOut, MetarouterPlan.Route calldata route)
    // ...
{
    // @audit same full-balance sizing on the leaf
    batch.amountIn = IERC20(tokenIn).balanceOf(address(this));
    require(batch.amountIn > 0, ZeroAmount());
    // ...
    batch.minAmountOut = route.kind != MetarouterPlan.SwapKind.None
        ? _minimumOut(tokenIn, tokenOut, route, batch.amountIn)
        : batch.amountIn;
    batch.metarouter = address(metarouter);
    // @audit the exact balance check is also derived from the nominal amount
    batch.balanceCheckMin = IERC20(tokenOut).balanceOf(address(metarouter)) + batch.minAmountOut;
}
```

A token that delivers less than reported by more than `maxSlippageBps` (which can only be tightened), or one that caps the size of a single transfer, can never satisfy the floor or the exact `BALANCE_CHECK`, so every harvest of it reverts. The project's unit test "drains fee-on-transfer inputs fully" passes only because the MetaRouter mock pays a fixed output. Third, both harvest paths unconditionally reset the allowance to zero after execution:

```
function _runHarvest(MetarouterPlan.Batch memory batch, uint256 deadline, uint256 value) internal {
    (bytes memory commands, bytes[] memory inputs) = MetarouterPlan.encodeBatch(batch);
    IERC20(batch.tokenIn).forceApprove(address(metarouter), batch.amountIn);
    _execute(commands, inputs, deadline, value);
    // @audit a token that rejects a zero approval reverts here on every harvest
    IERC20(batch.tokenIn).forceApprove(address(metarouter), 0);
}
```

`BaseCompounder.swapToAERO(...)` does the same after `_execute(...)`. A reward token whose `approve(spender, 0)` reverts or returns `false` can therefore never be harvested, even though the swap itself succeeded. All three triggers need either an Aero governance action (router delisting), a Timelock configuring an unusual bridge asset, or a non-standard reward token; plain ERC20 rewards are unaffected.

Impact: The affected token's reward inventory in the compounder is frozen with no recovery path. User principal and vault NAV are never touched; only the compounding yield held in that token is lost.

Recommendation(s): Add a Timelock-only `rescueERC20(token, to)` to both compounders that excludes AERO, which must keep flowing to the vault. Optionally let the executor pass a sell amount capped by the held balance and measure the output as a balance delta. Clear the allowance only when it is non-zero, or drop the redundant reset. Make the MetaRouter binding Timelock-updatable with the same constructor checks, or document the redeploy procedure for a router deprecation.

Status: Acknowledged

Update from the client: We acknowledge that router availability and token semantics can prevent compounding and strand affected reward inventory. We prefer to retain the current design. Future reward collectors can be redeployed or repointed, but rewards already trapped in an old contract may remain stranded because its router reference is immutable. This is a residual yield risk even where the deployment is outside the primary audit. We do not propose removing allowance resets as a general solution.

6.2.9 [Low] Compounder constructors do not cross-check the counterparties they bind immutably

File(s): `BaseCompounder.sol`, `BaseCompounder.sol`, `LeafCompounder.sol`

Description: Both compounders take a `Deployment` struct in the constructor and store its addresses as immutables. The constructors verify the MetaRouter against the factory registry and require a non-zero value for each field, but they do not check that the counterparties agree with each other, although the project enforces exactly this kind of reciprocity for the vault, Router and PositionVault in `LiquidityVault.finalizeDeployment(...)`. Three bindings are affected. `BaseCompounder` stores `cfg.aeroToken` and `cfg.liquidityVault` independently:

```
constructor(Deployment memory cfg) {
    require(
        cfg.governanceTimelock != address(0) && cfg.securityCouncil != address(0)
            && cfg.routeExecutor != address(0) && cfg.liquidityVault != address(0) && cfg.aeroToken != address(0)
            && cfg.metarouter != address(0) && cfg.factoryRegistry != address(0)
    );

    securityCouncil = cfg.securityCouncil;
    liquidityVault = ILiquidityVault(cfg.liquidityVault);
    // @audit never compared with the token the vault accounts
    aeroToken = IERC20(cfg.aeroToken);
    // ...
}
```

`_forwardAllAEROToVault()` transfers the compounder's token to the vault, discards the amount the vault reports as recognised and emits `AEROForwarded` regardless:

```
function _forwardAllAEROToVault() internal returns (uint256 amount) {
    amount = aeroToken.balanceOf(address(this));
    require(amount > 0, ZeroAmount());
    aeroToken.safeTransfer(address(liquidityVault), amount);
    // @audit the vault only recognises its own aeroToken; the return value is not checked
    liquidityVault.accountCompoundedRewards();
    emit AEROForwarded(amount);
}
```

`LiquidityVault.accountCompoundedRewards()` recognises only the surplus of its own `aeroToken`. With a mismatched token every forwarded reward lands in the vault unbooked, no transaction fails, and the vault has no rescue path for a foreign token, so the yield is lost for the life of the deployment. `LeafCompounder` copies `cfg.receiptToken` with no check and accepts any non-zero `baseCompounder`:

```
constructor(Deployment memory cfg) {
    require(
        cfg.governanceTimelock != address(0) && cfg.securityCouncil != address(0)
            && cfg.routeExecutor != address(0) && cfg.metarouter != address(0)
            && cfg.factoryRegistry != address(0) && cfg.baseCompounder != address(0)
    );
    require(cfg.baseDomain != 0);

    securityCouncil = cfg.securityCouncil;
    metarouter = IMetarouter(cfg.metarouter);
    factoryRegistry = IAeroFactoryRegistry(cfg.factoryRegistry);
    baseDomain = cfg.baseDomain;
    // @audit may equal the MetaRouter itself; the host then rejects every harvest and redeem
    baseCompounder = cfg.baseCompounder;
    // @audit never compared with the token the MetaRouter's REDEEM command actually burns
    receiptToken = cfg.receiptToken;
    require(metarouter.FACTORY_REGISTRY() == cfg.factoryRegistry);
    require(factoryRegistry.isMetaRouterApproved(cfg.metarouter));
    // ...
}
```

The `REDEEM` command carries no token: the MetaRouter burns its own `EMISSION_TOKEN`, so `harvestAndRedeem(...)` swaps into whatever `receiptToken` was configured while the redeem leg funds the burn from a different asset. If `baseCompounder` is set to the MetaRouter's own address, the host's `CrosschainLib` rejects the recipient on every `harvestToBase(...)` and `harvestAndRedeem(...)`, freezing the leaf's

whole reward balance. Each case requires a deployer error on an immutable against a rehearsed deployment script, but the vault-token case fails silently with no failing transaction to reveal it.

Impact: A mis-bound Base compounder silently strands every forwarded reward in the vault unbooked; a mis-bound leaf compounder cannot redeem or cannot harvest at all. Only compounding yield is affected, never user principal or NAV.

Recommendation(s): Add `aeroToken()` to `ILiquidityVault` and require `cfg.aeroToken == liquidityVault.aeroToken()` in the `BaseCompounder` constructor; require a non-zero return from `accountCompoundedRewards()` in `_forwardAllAEROToVault()`. In the `LeafCompounder` constructor, require `cfg.receiptToken == address(0) || cfg.receiptToken == metarouter.EMISSION_TOKEN()` and `cfg.baseCompounder != cfg.metarouter`. Add the `(baseDomain, baseCompounder)` pair to the manifest verification and run a minimal-value test harvest before enabling routes.

Status: Acknowledged

Update from the client: We recognize the consequences of incorrect immutable bindings and will validate the approved token, vault, emission token, domain and recipient manifest and perform a small end-to-end test before funding. No adversarial exploit with correct bindings has been demonstrated. We retain the design without a constructor change; deployment checks reduce, but do not eliminate, human error.

6.2.10 [Low] `disableBridgeAsset` succeeds and emits for an unconfigured key

File(s): `LeafCompounder.sol`

Description: `LeafCompounder` keeps a `bridgeAsset` mapping from a bridge token to the Hyperlane `TokenRouter` that `harvestToBase(...)` uses to send that token to Base. The Timelock configures a pair with `setBridgeAsset(asset, bridgeRouter)`, and the Security Council or the Timelock can retire one immediately with `disableBridgeAsset(asset)`, which is the emergency lever for a compromised or deprecated router:

```
function setBridgeAsset(address asset, address bridgeRouter) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(asset != address(0) && bridgeRouter != address(0), ZeroAddress());
    require(ITokenRouter(bridgeRouter).token() == asset, "Router token mismatch");
    bridgeAsset[asset] = bridgeRouter;
    // ...
    emit BridgeAssetConfigured(asset, bridgeRouter);
}

// ...
function disableBridgeAsset(address asset) external {
    require(msg.sender == securityCouncil || hasRole(DEFAULT_ADMIN_ROLE, msg.sender), Unauthorized());
    // @audit no check that a binding exists; a router address, address(0) or a stale key all succeed
    delete bridgeAsset[asset];
    // @audit the same event as a real removal, carrying only the key
    emit BridgeAssetDisabled(asset);
}
```

The function authenticates the caller and then deletes and emits unconditionally. The mapping is keyed by the token, but the setter's second argument and the configure event name the router, which invites passing the router address in an incident. Passing a router address, `address(0)` or an already-disabled token succeeds and emits the same `BridgeAssetDisabled` as a real removal, so the operator sees confirmation while the binding stays live and the next `harvestToBase(...)` still resolves the router:

```
function harvestToBase(
    // ...
) external payable onlyRole(ROUTE_EXECUTOR_ROLE) whenNotPaused nonReentrant returns (uint256 deliveredAmount) {
    // ...
    // @audit still returns the router the operator believed retired
    address bridgeRouter = bridgeAsset[bridgeToken];
    require(bridgeRouter != address(0), BridgeNotConfigured());
    // ...
}
```

The precondition is a trusted-role mis-key during an incident; no untrusted party can trigger it.

Impact: During an incident the next harvest still pays a router the operator believed retired, and the emitted event gives no signal that nothing was removed.

Recommendation(s): Require an existing binding before deleting, for example `require(bridgeAsset[asset] != address(0), BridgeNotConfigured());`, and include the removed router address in the `BridgeAssetDisabled` event.

Status: Acknowledged

Update from the client: Disabling an absent key leaves the requested disabled state in place. The event does not assert that a binding previously existed. Operators should identify the asset key and verify the resulting mapping as the postcondition.

6.2.11 [Low] Launch safety relies on an unenforced seed and launch sequence, and the zero-supply state has two more routes than the register lists

File(s): `LiquidityVault.sol`, `LiquidityVault.sol`, `LiquidityVault.sol`, `LiquidityVault.sol`, `KNOWN_ISSUES.md`

Description: Three register rows (EGR-1, "Zero-supply master restart" and TEL-1) rely on the same two deployment steps: governance raises the supply cap, bootstraps the master veNFT and pauses in one atomic Timelock batch, and then transfers exactly 10 whole tAERO to a dead address before unpausing. Neither step is enforced by the contracts. `bootstrapMasterVeNFT(...)` mints shares to the source and nothing else, and no function checks that a retained seed exists:

```
function bootstrapMasterVeNFT(address _source, uint256 _tokenId)
    external nonReentrant onlyGovernance whenNotPaused returns (uint256 shares)
{
    // ...
    uint256 masterAmount = positionVault.bootstrapMasterToken(_source, _tokenId);
    require(masterAmount >= minDepositAmount, BelowMinDeposit());

    // @audit no seed is minted or required; the dead-address transfer is a runbook step only
    shares = _mintDepositShares(masterAmount, valueBefore, supplyBefore, _source);
    // ...
}
```

The launch Timelock delay is zero (register row "Governance delay and cancellation authority"), so a mistaken or out-of-order batch executes immediately and leaves no review window. If the cap is raised in a separate transaction, a pre-bootstrap depositor who burns all but one wei of tAERO captures half of the bootstrapped stake above its own deposit. In our reproduction the attacker gains 249.5 AERO and the master source loses the same amount. The register describes one route into the zero-supply state: a fully funded request burns the last tAERO while a VPM deposit leaves stake in the master. We found two further routes. First, the last holder's cash redemption burns the remaining supply with no check that a claim survives, while the master stake and held fee backing stay in the vault:

```
function withdrawAERO(address _recipient, uint256 _amount, uint256 _tAEROToBurn, uint256 _feeBacking)
    external whenNotPaused onlyRouter withCircuitBreaker
{
    // ...
    tAEROToken.burnFrom(msg.sender, _tAEROToBurn); // @audit may burn the entire supply
    _transferLiquidAERO(_recipient, _amount);
    // ...
}
```

Second, any caller can run `lockExcessAERO(...)` on a vault with zero supply and move its leftover liquid backing into the master. Once `lockedAmount() > 0`, every mint reverts permanently:

```
function _requireShareBacking() internal view {
    if (address(positionVault) != address(0) && positionVault.lockedAmount() > 0) {
        require(tAEROToken.totalSupply() > 0, "Unbacked master stake"); // @audit no revival path
    }
}
```

Impact: Under the documented launch sequence, none of these states is reachable. The protection depends entirely on governance running the exact batch and seed transfer with no Timelock delay, and a single deviation either lets an attacker capture part of the bootstrap stake or freezes all deposits with no recovery function.

Recommendation(s): Enforce the seed on-chain, for example by minting the retained seed to the dead address inside `bootstrapMasterVeNFT(...)`, or by requiring `totalSupply() >= SEED` after every burn in `withdrawAERO(...)` and `finalizeVeNFTWithdrawal(...)`. Consider also adding a `totalSupply() > 0` guard to `lockExcessAERO(...)` and a Timelock-only revival path for a vault with zero supply.

Status: Acknowledged

Update from the client: We acknowledge the reliance on the documented atomic launch and mandatory retained seed. With that seed in place, ordinary users cannot burn the dead address's shares, so the additional zero-supply routes require the seed to be absent. We will clarify the deployment checks and separately address zero NAV with outstanding supply.

6.3 Informational

6.3.1 [Info] `maxSlippageBps` does not apply to CL routes, so `tightenMaxSlippageBps(...)` cannot bound the executor on them

File(s): `BaseCompounder.sol`, `BaseCompounder.sol`, `LeafCompounder.sol`, `DEPLOYMENT.md`

Description: The compounders convert reward tokens through MetaRouter routes chosen by the trusted `ROUTE_EXECUTOR_ROLE`. The register row "Compounder swap minimums" accepts that the executor chooses the minimum output, with the live quote acting as a sanity floor. `DEPLOYMENT.md` states that `maxSlippageBps` "controls swap output" and lists `tightenMaxSlippageBps(...)` among the emergency controls. However, `_minimumOut(...)` applies the slippage floor to V2 routes only. A CL route takes the executor's `minAmountOut` as given, and the only remaining check is that it is non-zero:

```
function _minimumOut(address tokenIn, address tokenOut, MetarouterPlan.Route calldata route, uint256 amountIn)
    internal view returns (uint256 minAmountOut)
{
    if (route.kind == MetarouterPlan.SwapKind.ClExactIn) {
        minAmountOut = route.minAmountOut; // @audit maxSlippageBps is never consulted
    } else {
        // ...
        uint256 slippageFloor = _slippageFloor(quoted);
        minAmountOut = route.minAmountOut;
        require(minAmountOut >= slippageFloor, SlippageExceeded());
    }
    require(minAmountOut > 0, ZeroAmount()); // @audit 1 wei passes on CL routes
}
```

The same code is present in `LeafCompounder._minimumOut(...)`. The contract NatSpec notes that Aero has no exact CL quoter, but the operational documentation presents the slippage setter as a general control.

Impact: A compromised or faulty executor can sell any reward balance through a CL route for close to nothing, and lowering `maxSlippageBps` during an incident has no effect on that path. The compounders are outside the primary scope and the executor is a trusted role, so the impact is limited to compounder reward value.

Recommendation(s): State in `DEPLOYMENT.md` and in the register that `maxSlippageBps` covers V2 routes only. Consider bounding CL routes with a Timelock-managed pool allowlist or a TWAP-based reference price, or disabling CL routes through a governance flag that the Security Council can switch off during an incident.

Status: Acknowledged

Update from the client: We agree that `maxSlippageBps` is a V2-only floor and that the operational documentation should say so explicitly. CL routes rely on a trusted executor's nonzero minimum and registered pools; the Council can pause the compounders. We propose correcting the documentation rather than adding a CL oracle solely for consistency.

7 Additional Notes

This section provides supplementary auditor observations on the code. These points were not reported as individual issues, but they are informative recommendations to improve the quality and maintainability of the codebase.

7.1 Rewards recognised after a write-down also vest at the retained rate

The TEL-10 record accepts that rewards which survive a write-down vest before the original `periodFinish`, because `_writeDownShortage(...)` keeps the prior `rewardRate`. The same retained rate also applies to rewards recognised after the write-down while the stream is still active: `_addReward(...)` adds `_amount / remainingTime` on top of the retained rate, so new rewards also vest early. The record does not discuss this case. We recommend that the owner confirm the acceptance covers it, or recompute `rewardRate` from the remaining unvested rewards inside `_writeDownShortage(...)` 1, 2.

Update from the client: We acknowledge that a new reward added after unpause to a still-active post-loss stream can share an accelerated pooled vesting outcome. We accept the accelerated availability of pooled backed rewards as intended, including after unpause. Additions while paused follow the separately established immediate-vesting policy. This acceptance does not cover unbacked reward promotion under the finding in Section 6.2.2.

7.2 A shortage that appears while the vault is already paused waits for the Security Council

The register row "Liquid shortage while already paused" (O-17) states that `checkSolvency(...)` writes down a shortage only when it starts the pause, and that a later shortage requires the Security Council to call `writeDownShortage(...)`. The `!paused()` condition gives no protection: a write-down during an existing pause can only move recorded backing closer to the real balance. Removing the condition, or running the write-down in the paused branch as well, would let anyone settle a shortage without waiting for the Security Council, and paused exits would no longer be priced against backing that does not exist 1.

Update from the client: We agree that a further shortage can remain uncrystallised while the vault is already paused, affecting the timing of recorded backing and exits. Council-only write-down while paused is intentional to allow backing to be restored first. Governance alone can unpause and we will not unpause with an unresolved shortage. Emergency exits remain possible while paused at recorded NAV until recovery or write-down; neither the no-unpause policy nor the restoration window ensures fairness. A stronger pre-pricing control would require separate review.